

Carrera: Ingeniería de Software con IA

MÓDULOS Y PAQUETES PARA MACHINE LEARNING CON PYTHON

Objetivo:

Al finalizar la sesión, el estudiante **analiza las características estadísticas** de un conjunto de datos mediante medidas de tendencia central, dispersión y forma, interpretando sus resultados para apoyar decisiones de preparación y análisis de datos en proyectos de Machine Learning con Python.



Sesión:

- Estructura de datos y estadística descriptiva
- Forma de la distribución
- Comparativa y visualización estadística
- Conexión con Machine Learning

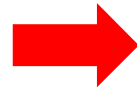
Vectores y matrices

Los datos deben representarse de forma que puedan ser procesados por algoritmos computacionales.

VECTOR

Representa los valores de UNA característica (ej. edad de 5 clientes)

22
25
31
28
35



MATRIZ

Varias características - filas = observaciones, columnas = features

Cliente	Edad	Ingreso	Compras
1	22	1500	2
2	25	1800	4
3	31	2500	7
4	28	2100	5
5	35	3200	9

NumPy y Pandas

Herramienta	Estructura	Función principal
NumPy	ndarray	Cálculo numérico, vectores, matrices y álgebra lineal
Pandas	DataFrame	Manipulación y análisis de datos tabulares

Si tenemos 1 000 clientes y 10 características, ¿cuántas filas y columnas tendrá la matriz de datos?

Respuesta: 1 000 filas y 10 columnas.

```
1 import numpy as np
2 import pandas as pd
3
4 edades = np.array(
5     [22, 25, 31, 28, 35]
6 )
7
8 print(edades)
9 print(edades.mean())
10
11 data = pd.DataFrame({
12     "Edad": [22, 25, 31, 28, 35],
13     "Ingreso": [1500, 1800, 2500,
14                2100, 3200],
15     "Compras": [2, 4, 7, 5, 9]
16 })
17
18 print(data)
19
```

```
[22 25 31 28 35]
28.2
      Edad  Ingreso  Compras
0       22     1500         2
1       25     1800         4
2       31     2500         7
3       28     2100         5
4       35     3200         9
```

Antes de construir un modelo, debemos de preguntarnos ...

1 ¿Cuál es el valor típico?

2 ¿Qué tan dispersos están los datos?

3 ¿Existen valores extremos?

4 ¿Los datos están concentrados o distribuidos?

5 ¿La distribución es simétrica?

6 ¿Existen posibles anomalías?

Media aritmética

Es simplemente el **promedio**.

$$\bar{x} = \sum x_i / n$$

Ejemplo:

20, 22, 24, 26, 28

$$\bar{x} = (20+22+24+26+28) / 5 = 24$$

La edad promedio es **24** años

```
1 import numpy as np
2 edades = np.array([20,22,24,26,28])
3 np.mean(edades)
4
```

24.0

¿Qué ocurre con un outlier?

20, 22, 24, 26, 100

$$\bar{x} = 38.4$$

Cuatro de los cinco valores están entre 20 y 26.

```
1 Q1 = data["edad"].quantile(0.25)
2 Q3 = data["edad"].quantile(0.75)
3
4 IQR = Q3 - Q1
5
6 limite_inferior = Q1 - 1.5 * IQR
7 limite_superior = Q3 + 1.5 * IQR
```



La media es sensible a valores extremos

Mediana

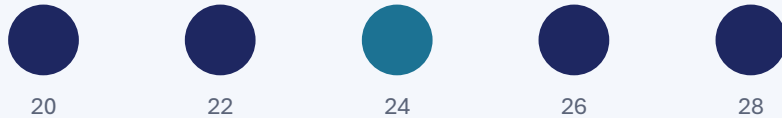
La mediana es el dato que ***queda en el centro y divide los datos en dos partes iguales.***

Sin outlier

20, 22, 24, 26, 28

Media = 24

Mediana = 24



Ambas medidas coinciden: la distribución es simétrica.

Con outlier

20, 22, 24, 26, 100

Media = 38.4 este desplazada

Mediana = 24 este resistente



La media cambió considerablemente; la mediana permaneció estable.

Mediana

La mediana es el dato que **queda en el centro y divide los datos en dos partes iguales**.

Cuando hay cantidad impar de datos

La posición de la mediana es:

$$\text{Mediana} = \frac{n + 1}{2}$$

Donde **n** es la cantidad de datos.



Tenemos: $n = 5$

Entonces: $\frac{5 + 1}{2} = 3$

La mediana está en la posición: **20**

Cuando hay cantidad par de datos

Se toman los dos valores centrales y se calcula su promedio

$$\text{Mediana} = \frac{\frac{X_n}{2} + \frac{X_n}{2} + 1}{2}$$

Ejemplo:



Tenemos: $n = 6$

Los valores centrales están en las posiciones: $\frac{6}{2} = 3$ y $3 + 1 = 4$

Entonces: $\text{Mediana} = \frac{20 + 25}{2} = 22.5$

Sensibilidad a outliers y aplicación

Medida	Sensibilidad a outliers
Media	Alta
Mediana	Baja

Aplicación: análisis de ingresos

1500 – 1600 - 1700 - 1800 - 20000

La media puede verse fuertemente afectada por el ingreso de 20 000. La mediana representa mejor el comportamiento típico.

Ejemplo:

1500 1600 1700 1800 1900 2100 2300 2500 3000 12000

¿Valor típico? Probablemente 1500 – 3000

¿La media representa bien al grupo? No necesariamente: 12000 la eleva

¿Qué ocurre con 12000? Es un posible valor extremo

¿Medida más robusta? La mediana

Varianza y desviación

La varianza mide cuánto se alejan los datos respecto de la media.

$$\sigma^2 = \sum (x_i - \mu)^2 / N$$

Varianza poblacional

$$\sigma = \sqrt{\sigma^2}$$

Desviación estándar

Ejemplo: 10, 12, 14

$\mu = 12$: distancias: -2, 0, +2, entonces al cuadrado: **4, 0, 4**

$$\sigma^2 = (4+0+4) / 3 = 2.67 \quad \sigma = \sqrt{2.67} \approx 1.63$$

Interpretación: una desviación estándar pequeña indica datos próximos a la media; una grande indica mayor dispersión.

```
1 datos = np.array([10, 12, 14])
2 np.var(datos)    # ddof=0 por defecto (poblacional)
```

2.6666666666666665

Varianza y desviación

La **varianza** mide cuánto se alejan los datos de la media, pero trabajando con las **diferencias al cuadrado**.

Ejemplo:

Tenemos: 2 4 6

Primero calculamos la media: $\bar{x} = \frac{2 + 4 + 6}{3} = 4$

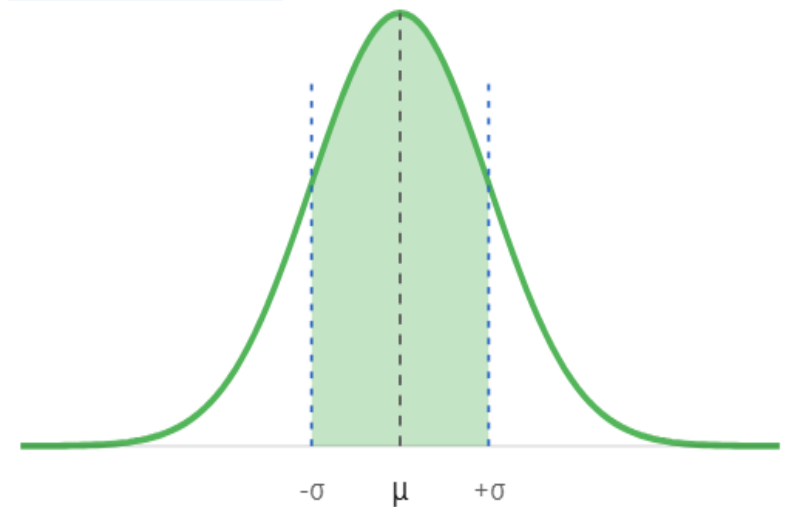
Ahora vemos cuanto se aleja cada dato de la media:

Dato	Media	Diferencia	Diferencia ²
2	4	-2	4
4	4	0	0
6	4	2	4

Sumamos: $4 + 0 + 4 = 8$

Dividimos entre la cantidad de datos: $\frac{8}{3} = 2.67$

$$\text{Var}(X) = 2.67$$



Varianza y desviación

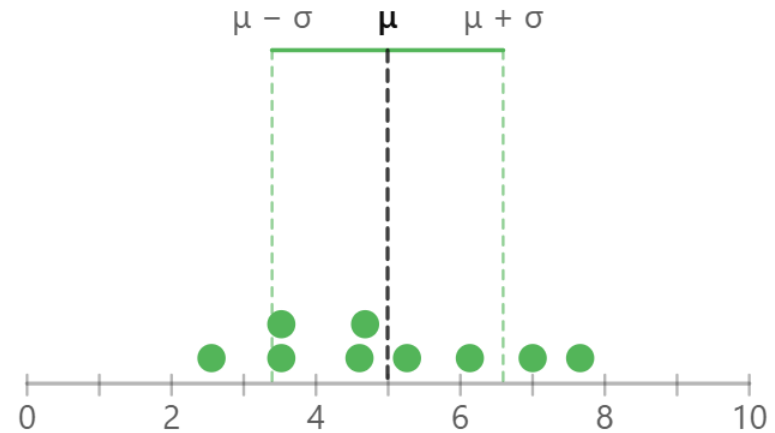
La **desviación estándar** es simplemente la **raíz cuadrada de la varianza**.

Ejemplo:

Del ejemplo anterior, la varianza es: 2.67

La desviación:

$$\sqrt{2.67} \approx 1.63$$



Entonces en Python sería:

```
1 import numpy as np
2
3 datos = [2, 4, 6]
4
5 np.var(datos)
```

2.6666666666666665

```
1 np.std(datos)
```

1.632993161855452

Varianza muestral: correlación de Bessel

Cuando tenemos solo una muestra de datos y queremos usarla para representar a toda una población, dividimos entre $n-1$ en lugar de n .

$$s^2 = \Sigma(x_i - \bar{x})^2 / (n - 1)$$

La corrección de Bessel hace que la *varianza muestral sea un estimador no sesgado de la varianza poblacional*.

En Python:

```
1 np.var(datos)           # poblacional (ddof = 0)
2 np.var(datos, ddof=1)   # muestral    (ddof = 1)
3 np.std(datos, ddof=1)   # desviación estándar muestral
4
```

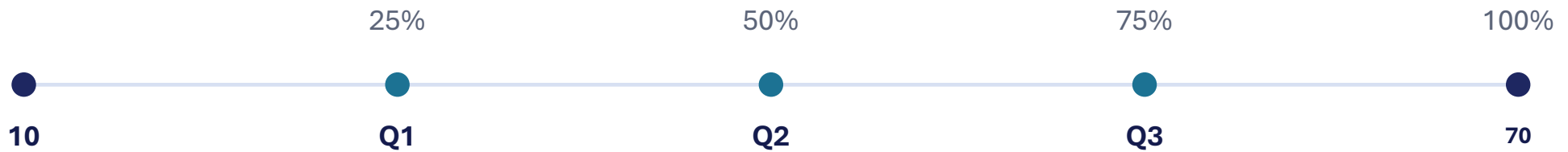
2.0

Cuartiles

Los **cuartiles** sirven para **dividir los datos ordenados en 4 partes**.

Cada cuartil nos indica hasta dónde llega el **25%, 50%, 75% y 100%** de los datos.

Ejemplo: 10, 20, 30, 40, 50, 60, 70



Los cuartiles permiten conocer cómo se distribuyen los datos.

```
1 np.percentile(datos, 25)
2 np.percentile(datos, 50)
3 np.percentile(datos, 75)
4
5.0
```

IQR y detección de outliers

Los cuartiles nos ayudan a encontrar el IQR, y el IQR nos ayuda a detectar posibles outliers.

¿Qué es el IQR?

IQR significa **Rango Intercuartílico** (*Interquartile Range*). Es la distancia que existe entre **Q1 y Q3**.

Ejemplo:

Tenemos: 10, 12, 14, 15, 16, 18, 20, 22, 24

Supongamos que obtenemos: $Q1 = 13$
 $Q3 = 21$

Entonces: $IQR = 21 - 13$
 $IQR = 8$

Ese **8** representa la distancia entre Q1 y Q3.

IQR y detección de outliers

Los cuartiles nos ayudan a encontrar el IQR, y el IQR nos ayuda a detectar posibles outliers.

¿Cómo usamos el IQR para encontrar outliers?

Usando dos límites:

$$\text{Límite inferior} \quad Q1 - 1.5(IQR)$$

$$\text{Límite superior} \quad Q3 + 1.5(IQR)$$

Del ejemplo anterior:

$$Q1 = 13$$

$$Q3 = 21$$

$$IQR = 8$$

Límite inferior

Límite superior



$$13 - 1.5(8) = 1$$

$$21 + 1.5(8) = 33$$

IQR y detección de outliers

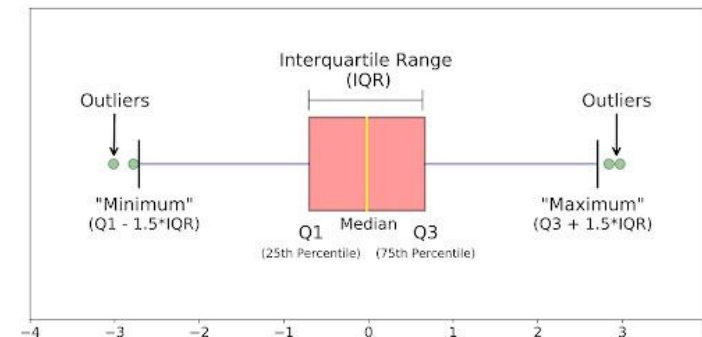
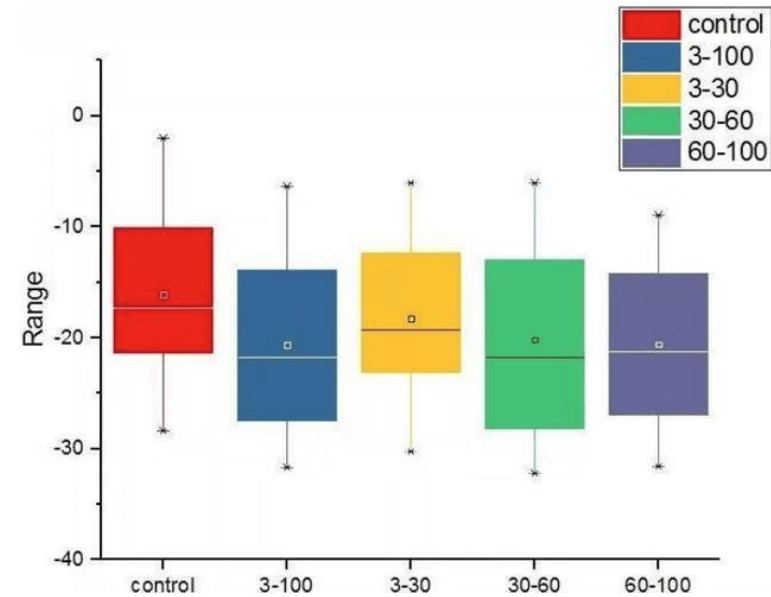
En Python

```
1 Q1 = data["ingresos"].quantile(0.25)
2 Q3 = data["ingresos"].quantile(0.75)
3
4 IQR = Q3 - Q1
5
6 limite_inferior = Q1 - 1.5 * IQR
7 limite_superior = Q3 + 1.5 * IQR
```

```
1 outliers = data[
2     (data["ingresos"] < limite_inferior) |
3     (data["ingresos"] > limite_superior)
4 ]
```

Podemos visualizarlo con un Boxplot

```
1 import seaborn as sns
2
3 sns.boxplot(x=data["ingresos"])
```



Deciles

Los **deciles** son parecidos a los **cuartiles**, pero en lugar de dividir los datos en **4 partes**, los dividen en **10 partes iguales**.



Ejemplo

Si **D9 = 5000**, significa que aproximadamente el 90 % de los datos se encuentra por debajo de 5000 y el 10 % restante está por encima. Esto permite analizar las colas de una distribución.

Deciles vs. cuartiles: misma lógica de posición, mayor granularidad y útiles para observar las colas extremas de la distribución (ejemplo: el 10 % de ingresos más altos).

Deciles

En Python:

```
1 data["ingresos"].quantile(0.1) # D1
2 data["ingresos"].quantile(0.2) # D2
3 data["ingresos"].quantile(0.3) # D3
```

```
1 deciles = data["ingresos"].quantile(
2     [0.1, 0.2, 0.3, 0.4, 0.5,
3     0.6, 0.7, 0.8, 0.9]
4 )
5
6 print(deciles)
7
```



Entonces:

Cuartil: 4 partes

Decil: 10 partes

Percentil: 100 partes

Coeficiente de variación (CV)

$$CV = (s / \bar{x}) \times 100$$

Permite comparar la variabilidad relativa entre variables con escalas diferentes (edad, ingreso, consumo, tiempo).

Donde:

- $s \rightarrow$ **desviación estándar muestral**
- $\bar{x} \rightarrow$ **media**
- $\times 100 \rightarrow$ para expresarlo como **porcentaje**

Variable A

Media = 100 · $\sigma = 10$

$$CV = 10\%$$

Variable B

Media = 20 · $\sigma = 5$

$$CV = 25\%$$

Aunque la desviación estándar de B (5) es menor que la de A (10), B presenta mayor variabilidad relativa: 25% vs 10%.

Precaución: no usar CV cuando la media es cero o muy cercana a cero; puede producir valores inestables.

Coeficiente de variación (CV)

Ejemplo:

Media = S/ 2,000

Desviación estándar = S/ 100

Entonces:

$$CV = \frac{100}{2000} \times 100$$

$$CV = 5\%$$

Ejercicio:

Media = S/ 10,000

Desviación estándar = S/ 1000

Entonces:

$$CV = \frac{100}{10000} \times 100$$

$$CV = 10\%$$

Coeficiente de variación (CV)

En Python:

```
1 media = data["ingresos"].mean()
2 desviacion = data["ingresos"].std()
3
4 cv = (desviacion / media) * 100
5
6 print(cv)
```

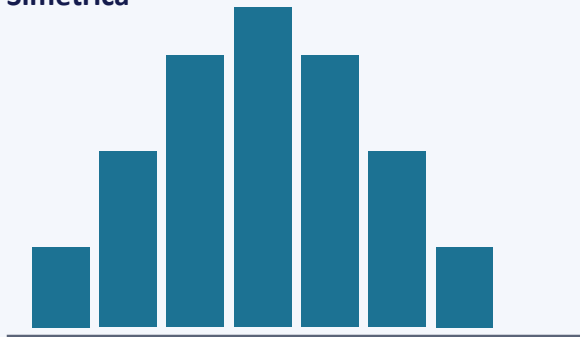
```
1 cv = data["ingresos"].std() / data["ingresos"].mean() * 100
```

El coeficiente de variación indica qué tan grande es la desviación estándar respecto a la media y se expresa normalmente como porcentaje.

Asimetría (Skewness)

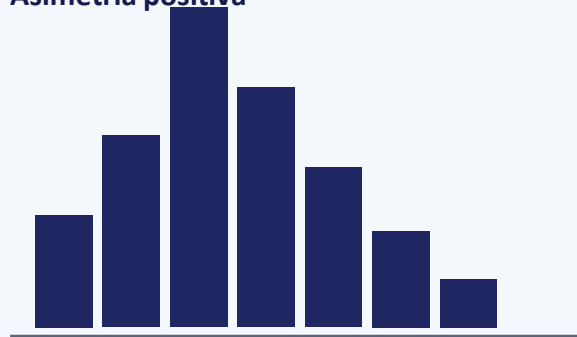
La asimetría indica si la distribución presenta una cola más prolongada hacia un lado.

Simétrica



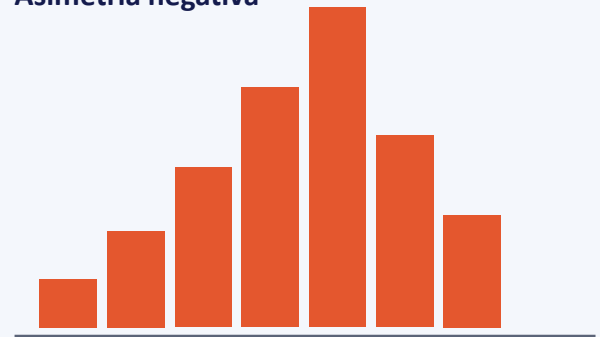
Skewness ≈ 0

Asimetría positiva



Skewness > 0

Asimetría negativa



Skewness < 0

Ejemplo típico: `ingresos.data["Ingreso"].skew() = 2.85` - La asimetría positiva considerable, cola hacia valores altos.

Media, mediana y asimetría

Distribución simétrica

$\text{Media} \approx \text{Mediana}$

Asimetría positiva

$\text{Media} > \text{Mediana}$ (generalmente)

Asimetría negativa

$\text{Media} < \text{Mediana}$ (generalmente)

Importante: no debe tomarse como una regla matemática absoluta para cualquier conjunto de datos, pero sirve como una excelente intuición inicial.

Curtosis

Describe la concentración y, especialmente, el comportamiento de las colas. Trabajaremos con curtosis en exceso:

$$K < 0$$

Platicúrtica - Positiva

Colas más ligeras

$$K \approx 0$$

Normal

Curtosis de referencia

$$K > 0$$

Leptocúrtica - Negativa

Colas más pesadas, mayor
posibilidad de valores extremos

`data["Ingreso"].kurt() → 4.2` : curtosis en exceso positiva: colas relativamente pesadas y mayor presencia potencial de valores extremos.

Puede indicar colas pesadas y mayor posibilidad de observar valores extremos

| Conectando los temas...

Ingresos mensuales: 1500, 1600, 1700, 1800, 1900, 2100, 2300, 2500, 3000, 12000



Hallar:

- Media
- Mediana
- Desviación estándar
- IQR

Prueba de Shapiro - Wilk

Evalúa estadísticamente si una muestra es compatible con una distribución normal. La normalidad puede ser relevante para ciertas técnicas y supuestos, pero no es un requisito general de todos los algoritmos de ML.

¿Mis datos se parecen a una distribución normal?

H0

Los datos son compatibles con una distribución normal.

H1

Los datos NO son compatibles con una distribución normal.

Regla de decisión ($\alpha = 0.05$)

$p < 0.05$ = Se rechaza H0

$p \geq 0.05$ = No se rechaza H0

No debemos decir “demostramos que los datos son normales”, solo que no hay evidencia suficiente para afirmar que se apartan de la normalidad.

Shapiro – Wilk

p-value = 0.32

No se rechaza H0

No hay evidencia suficiente de que los datos se aparten de la normalidad.

p-value = 0.001

Se rechaza H0

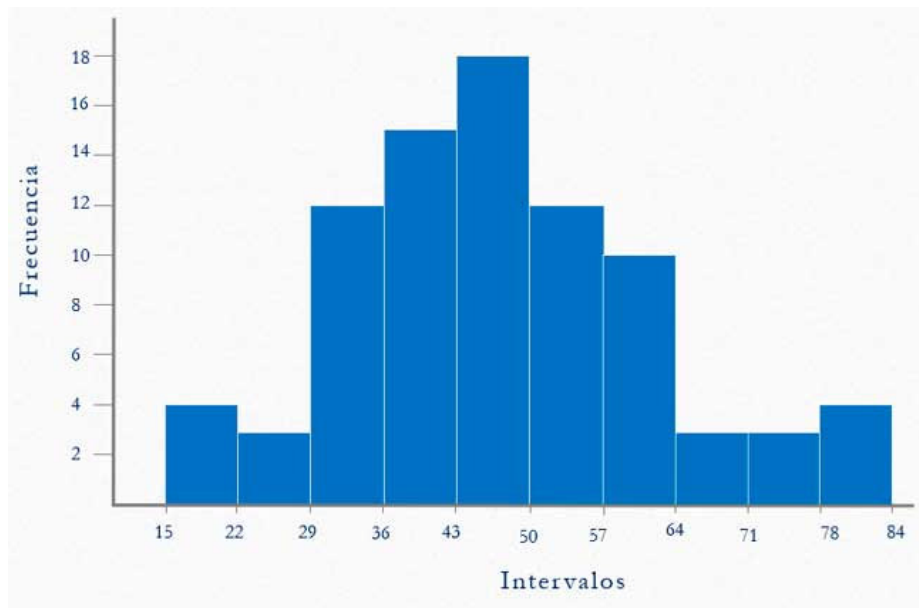
Existe evidencia de que los datos no siguen una distribución normal.

```
1 from scipy import stats
2 resultado = stats.shapiro(data["Ingreso"]) # statistic=0.8475, pvalue=0.0
3
```

Advertencia con muestras grandes: el test puede producir p-value ≈ 0 aun cuando la desviación de la normalidad no sea importante en la práctica. Por eso no debe usarse de forma aislada.

Histograma

El **histograma** nos permite ver **la forma de los datos**.



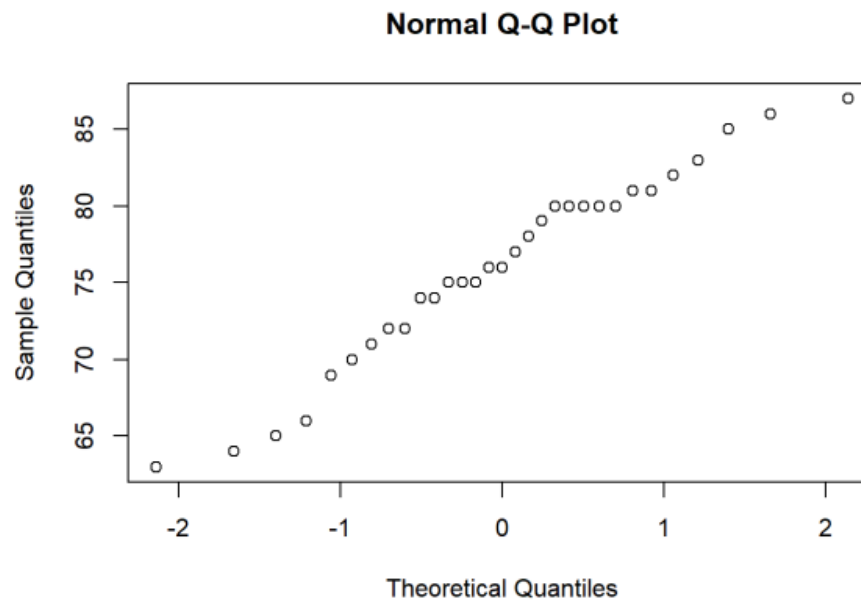
Es una **primera mirada** a los datos.

Debemos preguntarnos:

- ¿Tiene forma de campana?
- ¿Está centrado?
- ¿Tiene una cola muy larga?
- ¿Hay varios grupos?
- ¿Hay valores extremos?

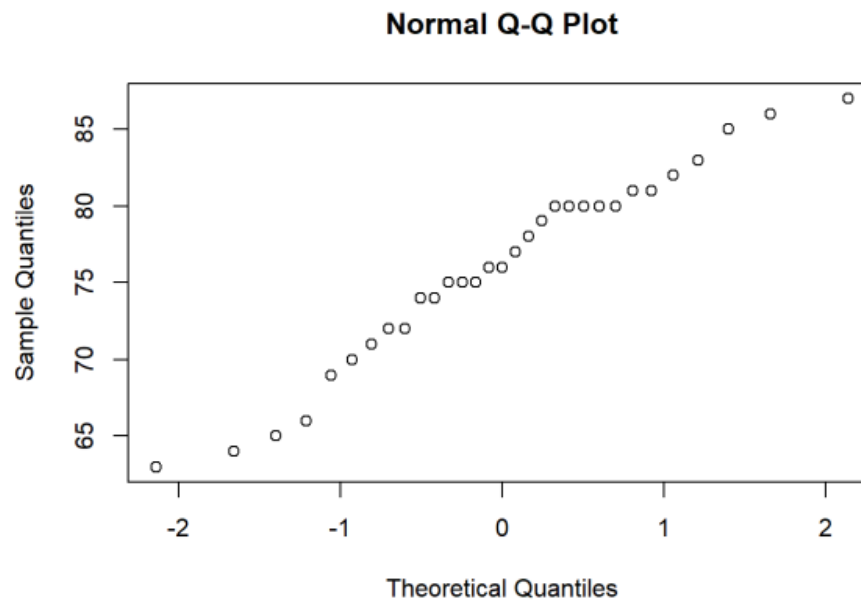
Q-Q Plot

Compara los cuantiles de nuestros datos con los cuantiles que esperaríamos si los datos siguieran una distribución normal.



Q-Q Plot

Compara los cuantiles de nuestros datos con los cuantiles que esperaríamos si los datos siguieran una distribución normal.



Comparativa de métricas

Métrica	¿Qué mide?	Sensibilidad a outliers	Aplicación
Media	Centro	Alta	Resumen general
Mediana	Centro	Baja	Datos asimétricos
Varianza	Dispersión	Alta	Análisis estadístico
Desviación estándar	Dispersión	Alta	Escalamiento/análisis
Q1 / Q3	Posición	Baja	Distribución
IQR	Dispersión central	Baja	Outliers
CV	Variabilidad relativa	Alta	Comparar variables
Skewness	Asimetría	Alta	Forma de distribución
Kurtosis	Colas	Alta	Valores extremos
Shapiro-Wilk	Compatibilidad c/ normalidad	Alta (muestras grandes)	Evaluación estadística

Cuatro formas de ver los datos

1 Histograma

```
data["Ingreso"].hist()
```

¿Dónde se concentran los datos? ¿Existe una cola?

2 Boxplot

```
data.boxplot(column="Ingreso")
```

Identifica Q1, mediana, Q3, IQR y posibles extremos.

3 Histograma + KDE

```
sns.histplot(data["Ingreso"], kde=True)
```

Visualiza distribución, concentración y asimetría.

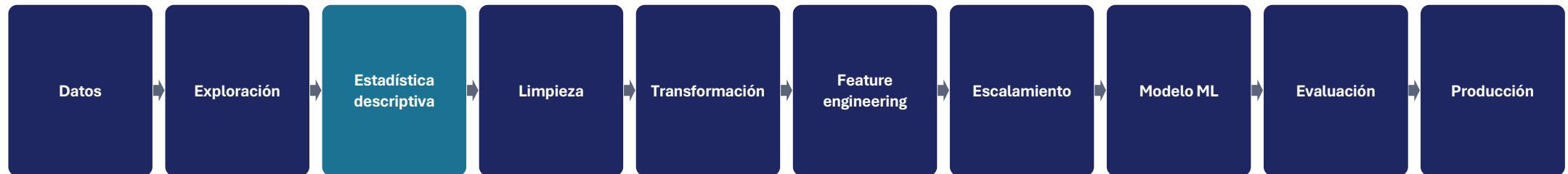
4 Q-Q Plot

```
stats.probplot(data["Ingreso"], dist="norm", plot=plt)
```

Evalúa visualmente la normalidad; complementa Shapiro-Wilk.

Un pipeline simplificado

Un **pipeline** es simplemente una **secuencia ordenada de pasos** que seguimos para transformar los datos y llegar a un resultado.



La estadística descriptiva es el puente entre los datos crudos y las decisiones de preparación que alimentan al modelo.

Idea fundamental

La estadística descriptiva ayuda a decidir qué transformación necesita cada variable, antes de que el modelo la reciba?

Outliers y transformación de variables

Un outlier no es necesariamente un error

Edad = 250 = Probablemente un error

Ingreso = 100 000 = Puede ser perfectamente real

Puede representar: error de digitación - fraude - evento excepcional - cliente especial - comportamiento real.

Feature Engineering - transformación logarítmica

$$X' = \log(1 + X)$$

```
data["Ingreso_log"] = np.log1p(data["Ingreso"])
```

Reduce la asimetría positiva fuerte. Recalcular `.skew()` para verificar el efecto.

Tabla de decisión

Encontramos	Pregunta	Posible acción
Muchos outliers	¿Errores o casos reales?	Investigar
Alta asimetría	¿Distribución sesgada?	Evaluar transformación
Diferentes escalas	¿Magnitudes muy distintas?	Escalamiento

Escalamiento de variables

Edad (18–70) e Ingreso (1000–15000) están en escalas muy diferentes.

StandardScaler

$$z = (x - \mu) / \sigma$$

```
1 from sklearn.preprocessing import StandardScaler
2
3 scaler = StandardScaler()
4 data["Ingreso_std"] = scaler.fit_transform(
5     data[["Ingreso"]]
6 )
7
8 from sklearn.preprocessing import StandardScaler
9
10 scaler = StandardScaler()
11
12 X_scaled = scaler.fit_transform(X)
```

Sensible a outliers, útil cuando la distribución es aproximadamente simétrica.

RobustScaler

Basado en mediana e IQR

```
1 from sklearn.preprocessing import RobustScaler
2
3 scaler = RobustScaler()
4 data["Ingreso_robust"] = scaler.fit_transform(
5     data[["Ingreso"]]
6 )
7
8 from sklearn.preprocessing import RobustScaler
9
10 scaler = RobustScaler()
11
12 X_scaled = scaler.fit_transform(X)
```

Recomendado cuando existen outliers importantes en la variable.

StandardScaler usa media y desviación estándar; RobustScaler usa mediana e IQR y es más resistente a los outliers.

| Data Drift

Una vez que el modelo está en producción, los datos pueden cambiar.

Entrenamiento

Ingreso medio = 2500

Skewness = 1.2

Producción (meses después)

Ingreso medio = 3200

Skewness = 2.8



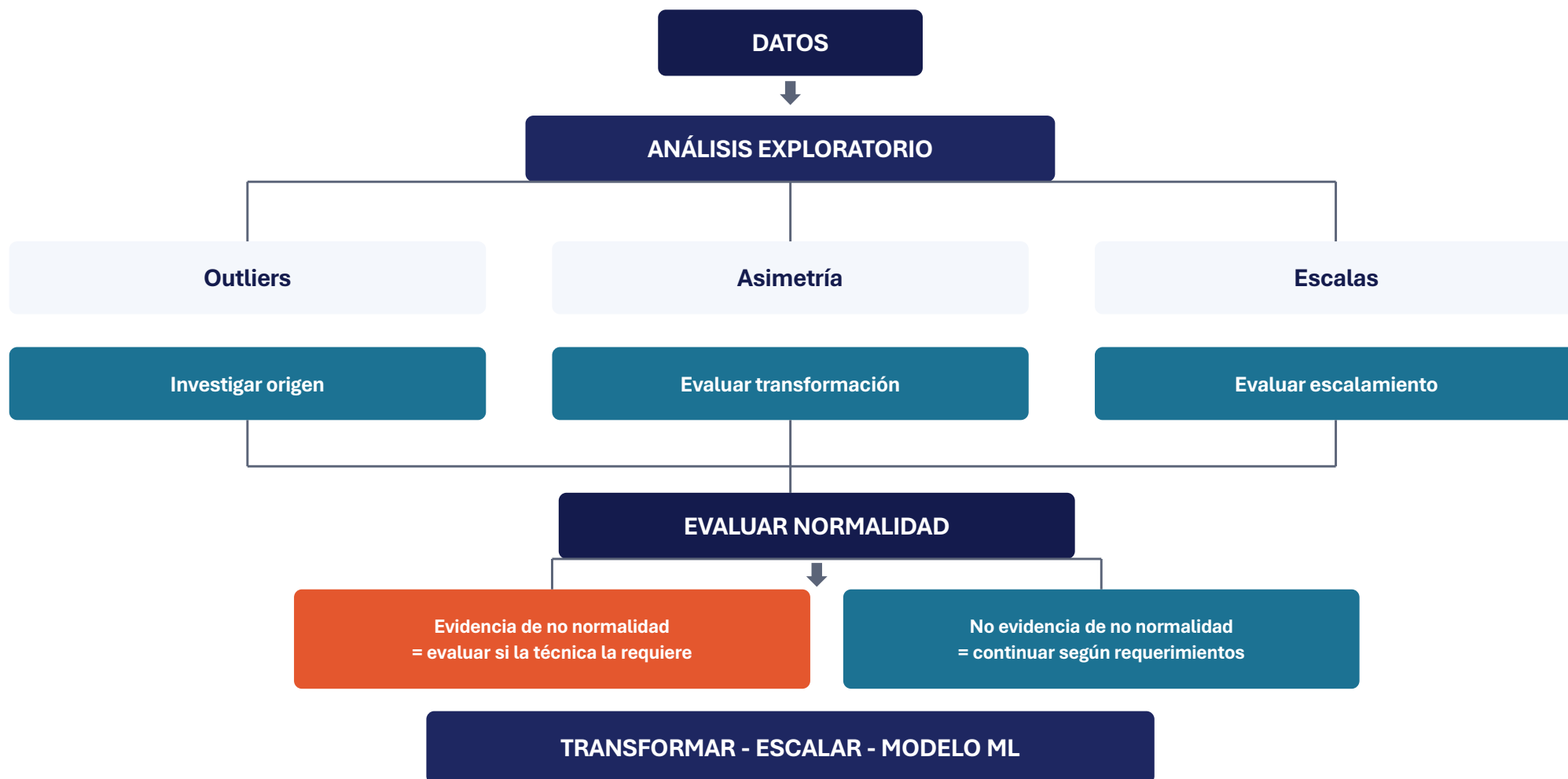
Importante: skewness y kurtosis pueden ayudar al monitoreo, pero no son suficientes por sí solas para determinar Data Drift.

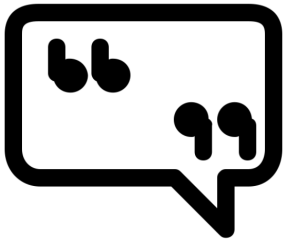
De la estadística a la decisión de ML



Encontramos	Pregunta	Posible acción
Muchos outliers	¿Son errores o casos reales?	Investigar
Alta asimetría	¿La distribución está sesgada?	Evaluar transformación
Diferentes escalas	¿Magnitudes muy diferentes?	Escalamiento
Alta variabilidad	¿La variable aporta información?	Analizar feature
Datos categóricos	¿Cómo representarlos?	Encoding
Valores faltantes	¿Qué información falta?	Imputación / eliminación
Data drift	¿Cambió la distribución?	Monitorear / reentrenar
Distribución no normal	¿El modelo lo requiere?	Evaluar alternativa / transformación

De los datos a la decisión de modelamiento





“Los sistemas de información no solo mueven datos... mueven ideas, decisiones y el futuro de las organizaciones”